QVAC Foundation Engineer — Assignment Report

Task 1

Scripts for GGUF conversion and quantization

```
python convert_hf_to_gguf.py ./models/llama32-files/ --outtype f16 \
    --outfile ./models/llama32-full.gguf &> conversion.log

./build/bin/llama-quantize ./models/llama32-full.gguf ./models/llama32-q4.gguf Q4_0 &>
quantization.log
```

First token latency

Data was extracted from the output of:

```
./build/bin/llama-bench -m ./models/llama32-q4.gguf -p 1 -o json
```

Metric	Value
avg_ns	74,033,020
avg_ns / 1,000,000	74.033 ms

Average token generation speed

Data was extracted from the output of:

```
./build/bin/llama-bench -m ./models/llama32-q4.gguf -n 10 -o json
```

Field	Value
n_gen	10
avg_ns	665,234,026
avg_ts	15.033192

There are two methods to calculate the speed:

Method	Result
1 / avg_ts	0.066519472 s = 66.519 ms
avg_ns / n_gen	66,523,403 ns = 66.523 ms

RAM usage during inference

Extracted from logs:

```
./build/bin/llama-cli -m ./models/llama32-q4.gguf -p "What is bitcoin?" \
    -r "<|im_end|>" -r "<|im_start|>" --log-verbose --log-file ./exec.log
```

Memory breakdown [MiB]	Self	Model	Context	Compute
Host	14,844	308	14,336	200
CPU_REPACK	1,512	1,512	—	—

Output logs

- Log: https://drive.google.com/file/d/1F2pphKUT92JR6-QBI7xrmhvDiUWkY0xe/view?usp=drive_link
- Log: https://drive.google.com/file/d/1-EXL-rHxlBekvaSoh0SEC-Ym87yKRvBe/view?usp=drive_link

Screenshot

See Task1 - Screenshot.png

Task 2

Output quality

From my experience:

Q4_0 often produces no answer or rephrases it. Q4_HQQ provided slightly different reasonable answers. However, without -n 100 limitation it has a tendency to looping.

First token latency

	CPU	GPU (NVIDIA GeForce RTX 3060)
First token latency	189.391 ms	66.17 ms

Average token generation speed

According to 2 methods of calculation:

Method	CPU	GPU (NVIDIA GeForce RTX 3060)
1 / avg_ts	191.367 ms	68.656 ms
avg_ns / n_gen	192.272 ms	68.656 ms

RAM usage during inference

Memory breakdown [MiB]	SelfTotal	ModelFree	ContextModel	ComputeContext
Host	16,524	1,988	14,336	200

Final comparison

Model	Speed	Size	Quality
Q4_0	66 ms	1828 MB	Low
Q4_HQQ	191 ms	1996 MB	Medium

Scripts for GGUF conversion and quantization

```
./build/bin/llama-quantize ./models/llama32-full.gguf ./models/llama32-q4_hqq.gguf Q4_HQQ &>
quantization.log

./build/bin/llama-cli -m ./models/llama32-q4_hqq.gguf -p "What is bitcoin?" \
    -r "<|im_end|>" -r "<|im_start|>" -n 100

./build/bin/llama-cli -m ./models/llama32-q4_hqq.gguf -p "What is bitcoin?" \
    -r "<|im_end|>" -r "<|im_start|>" \
    --cache-type-k q4_hqq --cache-type-v q4_hqq -n 100
```

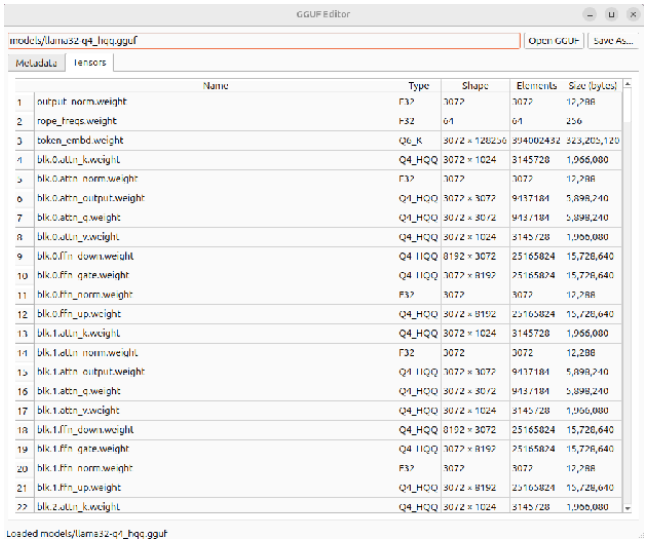
Update — July 29

ARM CPU (including NEON optimization) and Metal implementation was added and tested on MacBook Pro powered with Apple M1 and supporting Metal 3.

Gguf-py scripts were updated to work with Q4_HQQ quantisation. You can find in the Task2 folder results of:

```
python gguf-py/gguf/scripts/gguf_dump.py --json models/llama32-q4_hqq.gguf >
./llama32-q4_hqq.json

python gguf-py/gguf/scripts/gguf_editor_gui.py models/llama32-q4_hqq.gguf
```



I've run both python and native tests:

```
python gguf-py/tests/test_quants.py

cmake -B build -DLLAMA_BUILD_TESTS=ON
cmake --build build --config Release -j
./build/bin/test-backend-ops
```

Last week I made an attempt to implement the Vulkan version. But according to tests I found some problems with that. During implementation I got some reasonable answers, but after fixing the tests I've broken everything. ❗ So I kindly ask you not to try to run it.

Task 3

The flag can be used like:

```
./build/bin/llama-cli --list-devices
Available devices:
  CUDA0: NVIDIA GeForce RTX 3060 (11909 MiB, 11783 MiB free)

./build/bin/llama-cli -m model_name -mb CUDA0 ...
or
./build/bin/llama-cli -m model_name --mmproj-backend CUDA0 ...
```

Update — July 29

I've passed the mmproj-backend to the mtmd tool. The code was tested on the MacBook with the Pixtral-12b model. You can find the model files here:

- Model files: https://huggingface.co/bartowski/mistral-community_pixtral-12b-GGUF/tree/main

I have two possible backends on MacBook: **BLAS** and **MTL0**

The model was invoked like:

```
./build/bin/llama-cli -m ./models/mistral-community_pixtral-12b-IQ3_XXS.gguf \
    --mmproj ./models/mmproj-mistral-community_pixtral-12b-f16.gguf \
    --image ./s.jpg -p "What is on the picture?" \
    -r "<|im_end|>" -r "<|im_start|>" --no-mmap -ub 256 -c 8192 \
    -mb BLAS -dev MTL0
```

The tokens per second were:

-dev	-mb	Prompt	Generation
<not set>	<not set>	28.3	3.4
BLAS	<not set>	15.8	1.8
MTL0	<not set>	24.1	3.6
<not set>	BLAS	32.1	3.9
<not set>	MTL0	26.5	3.4
BLAS	BLAS	13.1	2.1
BLAS	MTL0	16.6	2.4
MTL0	BLAS	38.9	4.0
MTL0	MTL0	23.5	3.1

It seems that on my MacBook Metal is a good backend to run the text generation. But when it comes to image analysis it's better to use BLAS as a backed for the graphical part of the model.

I used to limit llama.cpp because I have a device with 8Gb RAM only.

Other

Environment

Task 1

I've already had llama.cpp downloaded and compiled. And even python. So the only modification to environment I had to do was python dependencies install:

```
pip install -e .

cmake -B build-d -DCMAKE_BUILD_TYPE=Debug
cmake --build build-d --config Debug
```

And these commands were executed before:

```
cmake -B build -DCMAKE_BUILD_TYPE=Release
cmake --build build --config Release
```

For machine with GPU NVIDIA GeForce RTX 3060:

```
cmake -B build -DGGML_CUDA=ON -DCMAKE_CUDA_ARCHITECTURES=86 \
    -DCMAKE_CUDA_HOST_COMPILER=/usr/bin/g++-11 \
    -DCMAKE_CUDA_COMPILER=/usr/local/cuda-12.8/bin/nvcc \
    -DGGML_NATIVE=OFF -DGGML_CUDA_FATTN=OFF

cmake --build build --config Release -j
```

Issues

Task 1

- Sometimes llama went to infinite looping or produced empty response.

Task 2

- It seems that the llama.cpp project was a little bit changed after you composed your task. At least src/llama-quantize.cpp is now src/llama-quant.cpp. But anyway chasing Q4_0 was the best advice.
- quantize_row_q4_0_impl has two ways of working. If it has no quant_weights parameter, it uses quantize_row_q4_0_ref, otherwise it has an algorithm using make_qx_quants function (as well as q3_K, q6_K and q5_0 quantization functions). I skipped the second branch for q4_hqq and always use quantize_row_q4_hqq_ref in quantize_row_q4_hqq_impl.
- I spent a lot of time trying to create an AVX2 implementation. Agents helped me a lot, but they are not able to do it directly. And I wasn't really familiar with AVX2. But finally I was happy to see 5 tokens per second instead of one.
- While compiling on a machine with CUDA I faced an issue with the 'movmatrix' feature support. It was strange because CUDA 12.8 was used. Anyway I have to switch off FATTN.

Suggestions

- I’d like to finish Vulkan version

- I’ve implemented only simple AVX2 and NEON optimizations. It would be great to have more complex solutions depending on the CPU versions as it’s done in Q4_0